

ZAD1

Zahaszuj słowo Linux za pomocą algorytmu MD5.

```
(.python_env) └─[admin@parrot]-[~]  
└─$ echo -n "Linux" | md5sum  
edc9f0a5a5d57797bf68e37364743831 -
```

```
(.python_env) └─[x]-[admin@parrot]-[~]  
└─$ echo -n "Linux" | openssl dgst -md5  
MD5(stdin)= edc9f0a5a5d57797bf68e37364743831  
(.python_env) └─[admin@parrot]-[~]
```

ZAD2

Zahaszuj słowo Linux za pomocą algorytmu SHA-256.

```
(.python_env) └─[admin@parrot]-[~]  
└─$ echo -n "Linux" | openssl dgst -SHA-256  
SHA2-256(stdin)= 4828e60247c1636f57b7446a314e7f599c12b53d40061cc851a1442004354fed  
(.python_env) └─[admin@parrot]-[~]  
└─$ echo -n "Linux" | sha256sum  
4828e60247c1636f57b7446a314e7f599c12b53d40061cc851a1442004354fed -  
(.python_env) └─[admin@parrot]-[~]  
└─$ echo -n "Linux" | openssl dgst -sha256  
SHA2-256(stdin)= 4828e60247c1636f57b7446a314e7f599c12b53d40061cc851a1442004354fed  
(.python_env) └─[admin@parrot]-[~]
```

SPRAWDZANIE JAKI HASH ZOSTAŁ UŻYTY

NIE BĘDZIE ZADAŃ Z PROGRAMOWANIEM

ZAD3

Zakoduj słowo Linux za pomocą kodowania base64.

```
(.python_env) └─[admin@parrot]-[~]  
└─$ echo -n "Linux" | base64  
TGluZXg=
```

Dodatkowo:

```
└─$ echo -n "Linux" | openssl dgst -md5
MD5(stdin)= edc9f0a5a5d57797bf68e37364743831
(.python_env) └─[admin@parrot]-[~]
└─$ echo -n "edc9f0a5a5d57797bf68e37364743831" | base64
ZWRjOWYwYTVhNWQ1Nzc5N2JmNjh1MzczNjQ3NDM4MzE=
```

ZAD4

Odkoduj słowo VU1DUw== zakodowane za pomocą base64.

```
└─$ echo -n "VU1DUw==" | base64 -d
UMCS(.python_env) └─[admin@parrot]-[~]
```

ZAD5

Zaszyfruj słowo Linux za pomocą algorytmu AES-256-ECB i klucza

E592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e.

-hex 32 bo $256/8 = 32$

Do sprawdzenia algorytmu: openssl (szukać cipher commands)

Tryb cbc potrzebuje wektora inicjalizacyjnego, a ecb nie

```
(.python_env) └─[admin@parrot]-[~]
└─ $openssl rand -hex 32
89a0ae457cf88d95b3f8b42b575a888ce1c6f79385690642b06c26609a1214704
(.python_env) └─[admin@parrot]-[~]
└─ $echo -n "Linux" > word.txt
(.python_env) └─[admin@parrot]-[~]
└─ $openssl enc -aes-256-ecb -in word.txt -out slowo.enc -K E592bc9e5fa8618a02edf437bdf3ffd4
fcdfbe9e588749db898a3662b461a80e
(.python_env) └─[admin@parrot]-[~]
└─ $cat slowo.enc
XXXXXXXXXXXXXXXXpython_env) └─[admin@parrot]-[~]
(.python_env) └─[*]-[admin@parrot]-[~]
└─ $openssl enc -aes-256-ecb -in word.txt -out slowo.enc -K E592bc9e5fa8618a02edf437bdf3ffd4
fcdfbe9e588749db898a3662b461a80e -a
(.python_env) └─[admin@parrot]-[~]
└─ $cat slowo.enc
qN3bUtgJx7jHygE8d8QlQ==
(.python_env) └─[admin@parrot]-[~]
└─ $openssl enc -aes-256-ecb -in word.txt -out slowo.enc -K E592bc9e5fa8618a02edf437bdf3ffd4
fcdfbe9e588749db898a3662b461a80e -base64
(.python_env) └─[admin@parrot]-[~]
└─ $cat slowo.enc
qN3bUtgJx7jHygE8d8QlQ==
```

odszyfrowanie (słowo.enc nie może być w base64 chyba że na końcu odszyfrowania też damy base64)

```

└─$ openssl enc -aes-256-ecb -in word.txt -out slowo.enc -K E592bc9e5fa8618a02edf437bdf3ffd4
fcdfbe9e588749db898a3662b461a80e
(.python_env) └─[admin@parrot]-[~]
└─$ openssl enc -d -aes-256-ecb -in slowo.enc -out slowo.dec -K E592bc9e5fa8618a02edf437bdf3
ffd4fcdfbe9e588749db898a3662b461a80e
(.python_env) └─[admin@parrot]-[~]
└─$ openssl enc -d -aes-256-ecb -in slowo.enc -out slowo.dec -K E592bc9e5fa8618a02edf437bdf3
ffd4fcdfbe9e588749db898a3662b461a80e
(.python_env) └─[admin@parrot]-[~]
└─$ cat slowo.dec
Linux(.python_env) └─[admin@parrot]-[~]

```

ZAD6

Zaszyfruj słowo Linux za pomocą AES-256-CFB, klucza

e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e oraz IV

83c468dc477543a6906912b6a1344416.

```

(.python_env) └─[admin@parrot]-[~]
└─$ openssl enc -AES-256-CFB -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461
a80e -iv 83c468dc477543a6906912b6a1344416 -in slowo.txt -out slowo.enc -a
(.python_env) └─[admin@parrot]-[~]
└─$ cat slowo.enc
eoBklRQ=
(.python_env) └─[admin@parrot]-[~]
└─$ openssl enc -d -AES-256-CFB -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b
461a80e -iv 83c468dc477543a6906912b6a1344416 -in slowo.enc -out slowo.dec -a
(.python_env) └─[admin@parrot]-[~]
└─$ cat slowo.dec
Hello(.python_env) └─[admin@parrot]-[~]
└─$

```

ZAD7

Zaszyfruj słowo Linux za pomocą AES-256-CFB, klucza

e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e oraz IV

83c468dc477543a6906912b6a1344416. Wynik szyfrowania zakoduj za pomocą kodowania base64.

ZAD8

Słowo f6NB8w== zostało zaszyfrowane algorytmem AES-256-CFB, kluczem

e592bc9e5fa8618a02edf437bdf3ffd4fcdfbe9e588749db898a3662b461a80e oraz IV

83c468dc477543a6906912b6a1344416, a następnie zakodowane za pomocą kodowania base64.

Odszyfruj i odkoduj słowo.

```
(.python_env) [admin@parrot]~]
└─ $echo -n "f6NB8w==" | base64 -d > slowo.txt
(.python_env) [admin@parrot]~]
└─ $cat slowo.txt
❖❖(.python_env) [admin@parrot]~]
└─ $openssl enc -d -AES-256-CFB -K e592bc9e5fa8618a02edf437bdf3ffd4fcdfe9e588749db898a3662b461a80e -iv 83c468dc477543a6906912b6a1344416 -in slowo.txt -out slowo.dec
(.python_env) [admin@parrot]~]
└─ $cat slowo.dec
MFI
(.python_env) [admin@parrot]~]
└─ $
```

Najpierw odkodować base64, potem odszyfrować

ZAD9

Wygeneruj klucz szyfrujący o długości 128 bitów.

```
(.python_env) [admin@parrot]~]
└─ $openssl rand -hex 16
96feb9cb54031e1d77daa60c6419ff87
```

ZAD10

Zaszyfruj słowo Linux za pomocą AES-128-CBC, bez soli, hasło do generowania klucza to

Ubuntu, funkcja generowania klucza to PBKDF2, z liczbą iteracji równą 256.

```
(.python_env) [admin@parrot]~]
└─ $echo -n "Linux" > slowo.txt
(.python_env) [admin@parrot]~]
└─ $openssl enc -AES-128-CBC -nosalt -in slowo.txt -out slowo.enc -pbkdf2 -iter 256 -a -pass pass:Ubuntu
(.python_env) [admin@parrot]~]
└─ $cat slowo.enc
gpIb5SsCn+wwuKhTOCHAqw==
(.python_env) [admin@parrot]~]
└─ $openssl enc -d -AES-128-CBC -nosalt -in slowo.enc -out slowo.dec -pbkdf2 -iter 256 -a -pass pass:Ubuntu
(.python_env) [admin@parrot]~]
└─ $cat slowo.dec
Linux(.python_env) [admin@parrot]~]
└─ $
```

Przy symetrycznym nie musimy generować klucza i wektora

Szyfrowanie asymetryczne V

ZAD11

Pliku.

ZAD12

pliku. Zaszzyfruj słowo Linux kluczem publicznym i odszyfruj prywatnym. Użyj paddingu OAEP.

```
(.python_env) ㉿[admin@parrot]-[~]
└─ $openssl pkeyutl -encrypt -pubin -inkey public.pem -in slowo.txt -out slowo.enc -pkeyopt
rsa_padding_mode:oaep
(.python_env) ㉿[admin@parrot]-[~]
└─ $openssl pkeyutl -decrypt -inkey keys.pem -in slowo.enc -out slowo.dec -pkeyopt rsa_paddi
ng_mode:oaep
(.python_env) ㉿[admin@parrot]-[~]
└─ $cat slowo.dec
Linux(.python_env) ㉿[admin@parrot]-[~]
└─ $
```

ZAD13

Wygeneruj parę kluczy (publiczny i prywatny), klucze mają 2048 bitów. Wyeksportuj klucze do

pliku. Zapisz do pliku słowo Linux. Podpisz plik używając klucza prywatnego, z paddingiem PSS:

hash SHA-256, długość soli powinna odpowiadać długości funkcji skrótu. Zweryfikuj wygenerowany podpis.

```
(.python_env) [admin@parrot]-[~]  
└─$ openssl genpkey -algorithm RSA -out keys.pem -pkeyopt rsa_keygen_bits:2048  
  
(.python_env) [admin@parrot]-[~]  
└─$ openssl pkey -in keys.pem -pubout -out public.pem
```

```
(.python_env) [x]-[admin@parrot]-[~]
└─ $openssl dgst -sha256 -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -sign keys.
pem -out signature slowo.txt
(.python_env) [admin@parrot]-[~]
└─ $cat signature
Y4 000000 00( 000000 f 00 I 000000 UX?L 000000 00DC 000000[h 000000 00K 000000 00G' 000000
/]w 000000( 000000 fJn 000000 APg 000000 000000 00 00c. 000000ç 000000 j 000000^ 000000w 000000 000000 00'q 000000C 000000'3H 000000
6 000000 000000>6 000000à 0000000000/h 000000(.python_env) [admin@parrot]-[~]
```

```

└─ $cat signature | base64
WTSzQ4emi+Qdji2DRzAovVYd8/kqnH5m1DNfSbz5T+gwVVgBP0zwkhtP7JmMVQjwSN72kN8YQNNC
MEQEQwnEMbKljx0G6DhbaPEsm++nA2uD3RdbHBMe16rep0vtAyLD12AOS4vdUJ6rRxNHYPpTum+9
L113r50JFFSkUSiQgIsrnpmpmSsmz/KkgQVBn67MXNJSN08kw0lK9linlaqgHJGMui0+8xPY90oGP
/7EuavrSeV7g3PERT4J3vNZCEfiwRLmdr7Mtr1EncadQUbo9Q6CVSyczAEi46Nq89cvrhcuXo+7g
NiAU9K79aD42ubw6w6DkEeLB1Yupmv20L2i45A==
(.python_env) └─[admin@parrot]─[~]
└─ $openssl base64 -in signature -out signature.b64
(.python_env) └─[admin@parrot]─[~]
└─ $cat signature
Y4 0000 00( 000 00f 00_I 000 0UX?L 000 000DC 0000 [h 00 00 00 00K 00'x'K 00 00G' 00 0
/]w 00000( 000 00fJn 00Apg 00 00 00 00 00 00c. 000 00ç 000 j 00^ 00000 00 00 0000 00'q 0Q 0C 00'3H 00
6 000 000 000>6 00 à 000 0000'h 00(.python_env) └─[admin@parrot]─[~]
└─ $cat signature.b64
WTSzQ4emi+Qdji2DRzAovVYd8/kqnH5m1DNfSbz5T+gwVVgBP0zwkhtP7JmMVQjw
SN72kN8YQNNCMEQEQwnEMbKljx0G6DhbaPEsm++nA2uD3RdbHBMe16rep0vtAyLD
12AOS4vdUJ6rRxNHYPpTum+9L113r50JFFSkUSiQgIsrnpmpmSsmz/KkgQVBn67MX
NJSN08kw0lK9linlaqgHJGMui0+8xPY90oGP/7EuavrSeV7g3PERT4J3vNZCEfiw
RLmdr7Mtr1EncadQUbo9Q6CVSyczAEi46Nq89cvrhcuXo+7gNiAU9K79aD42ubw6
w6DkEeLB1Yupmv20L2i45A==
(.python_env) └─[admin@parrot]─[~]
└─ $openssl dgst -sha256 -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -verify pub
lic.pem -signature signature slowo.txt
Verified OK

```

ZAD14

Złam hash MD5: e2fc714c4727ee9395f324cd2e7f331f za pomocą:

- 1) ataku słownikowego,
- 2) generując swój własny słownik, widząc, że hasło ma 4 znaki i składa się z małych liter a-z
- 3) za pomocą maski, widząc, że hasło ma 4 znaki i składa się z małych liter a-z

ZAD15

Wygeneruj parę kluczy GPG dla maila student@umcs.pl. Klucz RSA, 1024 bity.

Wyeksportuj klucz

publiczny i prywatny do pliku.

ZAD16

Zaszyfruj słowo Linux algorytmem AES z kluczem 128 bitów, z hasłem Ubuntu używając GPG.

Wynik zapisz do pliku. Odszyfruj plik.

ZAD17

Wygeneruj parę kluczy RSA (GPG) z domyślnymi parametrami. Zaszyfruj słowo Linux przy

użyciu klucza publicznego. Wynik szyfrowania zapisz do pliku. Odszyfruj plik.

ZAD18

Wygeneruj parę kluczy RSA (GPG) z domyślnymi parametrami. Zapisz słowo Linux do pliku,

podpisz plik przy użyciu klucza publicznego. Użyj kodowania base64. Wynik podpisywania zapisz

do pliku. Zweryfikuj podpisany plik.

ZAD19

Uruchom serwer i przetestuj wysyłanie do serwera pliku / tekstu:

```
podman run -p 5000:5000 docker.io/mazurkatarzynaumcs/echoserver:latest
```

```
docker run -p 5000:5000 docker.io/mazurkatarzynaumcs/echoserver:latest
```

